

Design Rationale – University Lost and Found

Bridge + Adapter Design Patterns

1. System Context

University Lost and Found is a Java application for processing lost and found item requests. A request can be either `LostItemRequest` or `FoundItemRequest`. The same request logic can work with different tracking mechanisms: `DatabaseTrackingService`, `QRTrackingService`, or the existing `OldLostFoundSystem` through `LegacySystemAdapter`. The user chooses the request type and tracking service at runtime.

2. Why the Bridge Pattern Is Used

The Bridge pattern separates the request abstraction from the tracking implementation. `LostFoundRequest` is the abstraction, while `LostItemRequest` and `FoundItemRequest` are refined abstractions. `TrackingService` is the implementor interface, and `DatabaseTrackingService` and `QRTrackingService` are concrete implementations. `LostFoundRequest` stores a `TrackingService` reference and delegates tracking work to it.

This separation is useful because request types and tracking services can vary independently. For example, a lost item can use the database service, while a found item can use the QR service, without creating classes such as `LostItemDatabaseRequest` or `FoundItemQRRequest`. This avoids a large combination of subclasses and keeps the design easier to extend.

3. Why the Adapter Pattern Is Also Needed

The Adapter pattern solves a different problem. `OldLostFoundSystem` is a legacy API and does not implement `TrackingService`. The new application expects `registerItem(String itemName, String location)` and `findItem(String itemId)`, but the legacy class exposes `saveOldRecord(int locationCode, String itemName)` and `searchByNumericCode(int itemCode)`. Its parameter types, method names, return values, and error conventions are different.

`LegacySystemAdapter` implements `TrackingService` and translates between these incompatible APIs. It converts `String` location and item IDs to integers, calls the legacy methods, converts legacy error codes or null results into `IllegalArgumentException`, and returns results in the format expected by the new system. Therefore, the legacy implementation can participate in the Bridge without changing `OldLostFoundSystem`.

4. How Bridge and Adapter Work Together

Bridge and Adapter have separate responsibilities. Bridge provides flexibility inside the new design by allowing `LostFoundRequest` to work with any `TrackingService` implementation. Adapter provides compatibility at the system boundary by making `OldLostFoundSystem` look like a `TrackingService`. `LegacySystemAdapter` therefore becomes another valid Bridge implementor alongside `DatabaseTrackingService` and `QRTrackingService`.

5. Complexity Module: Dynamic Implementor Selection

The project uses dynamic implementor selection. `TrackingServiceSelector.select(String type)` chooses `DatabaseTrackingService`, `QRTrackingService`, or `LegacySystemAdapter` at runtime. Main obtains the user's service choice and passes the selected `TrackingService` to `LostItemRequest` or `FoundItemRequest`. The client request classes do not hard-code a concrete tracking implementation.

6. Trade-off / Limitation

A limitation is that `TrackingServiceSelector` contains the mapping between text values and concrete implementations. Adding a new tracking service currently requires modifying the selector. In addition, `LegacySystemAdapter` assumes that legacy locations and item IDs are numeric, so non-numeric legacy values are rejected. The benefit is that this compatibility logic stays isolated in the adapter instead of spreading through the request classes.

7. Verification

JUnit 5 tests verify Bridge behavior with database and QR implementations, Adapter registration and invalid-location handling, and runtime selection of database, QR, and legacy services, including an unknown-service error case. The current test suite contains eight passing tests.